

Lab 07 – OFDM Simulation

Students may collaborate with other members of their group in writing the simulation software, but not with members of other groups. As with all lab reports, each student must write up his/her own lab report, without collaborating with others.

Objectives

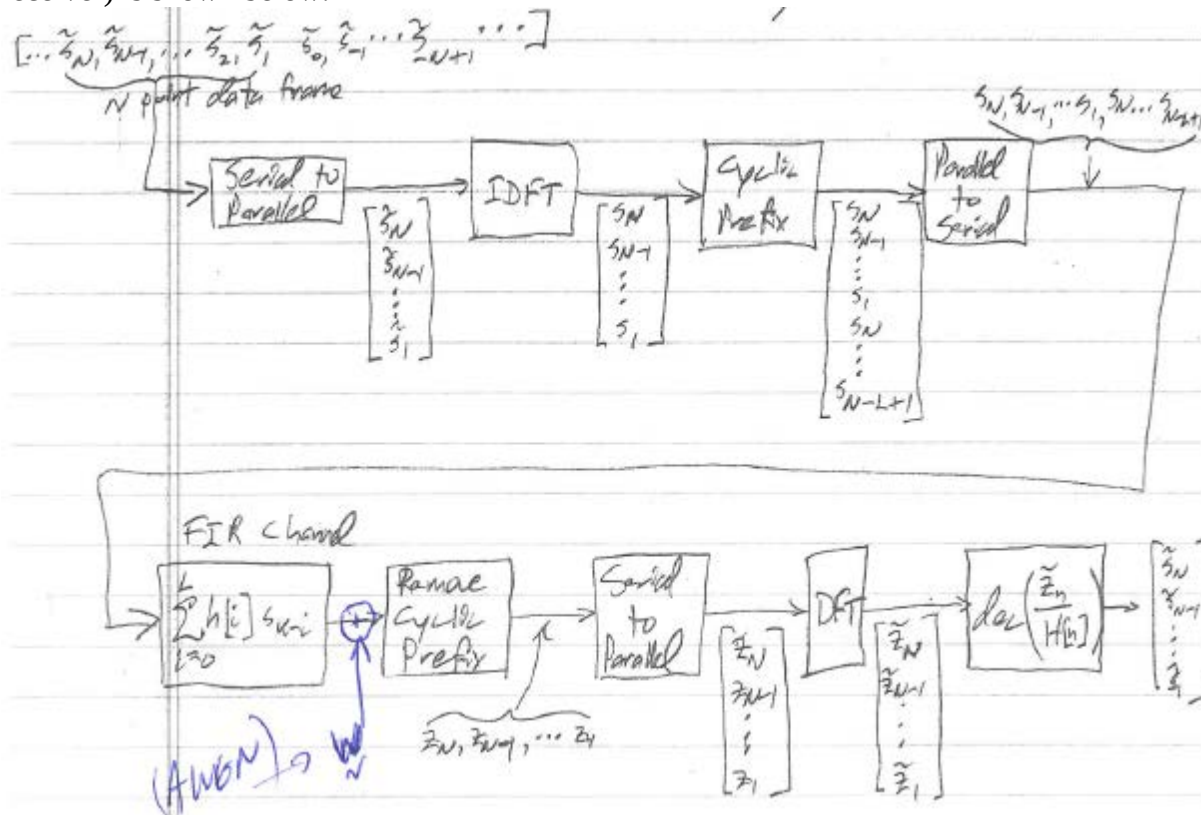
- Simulate a 16 channel OFDM system over a discrete-time finite impulse response (FIR) linear filter channel with AWGN, using BPSK modulation in all channels.
- Compare received SNRs and BERs over the 16 channels.

Equipment needed

- Computer running MATLAB.

Background

Orthogonal frequency division multiplexing (OFDM) is the multiple access technique used in modern (G4) cellular telephone systems; variations of OFDM are being proposed for G5 systems as well. OFDM employs cyclic prefix insertion and frequency-domain equalization to greatly simplify the task of equalizing the intersymbol interference (ISI) that occurs on frequency selective channels. A block diagram of an OFDM system (transmitter, FIR channel, and receiver) is shown below.



Procedure and Analysis

- Follow the steps below to fill in the missing sections in the MATLAB code at the end of this lab assignment, which implements a 16 channel BPSK OFDM simulator. This program has step-by-step comments telling you what to do, but you need to fill in the missing MATLAB code by replacing the question marks with one or more lines of code. There is more than one way to fill in the missing code, and anything that works will earn full credit. You should refer to the OFDM lecture notes (especially the block diagram on page 10), and the OFDM reading assignment when doing this lab. *Make sure to turn in your Matlab code with your lab report.*
- (a) Fill in the part of the code that computes the channel frequency response vector H . Turn in a printout of the numbers in H , and also the two stem plots of $\text{abs}(H)$ and $\text{angle}(H)$. You can comment off the rest of the code, or delete it (after saving a copy of it). It is recommended that you run the check included in the code to make sure that the forward and inverse DFT matrices multiply to the 16×16 identity matrix, but you don't need to turn in the results of the check.
- (b) Fill in the code through the line where the variable "errors" is computed. Turn in printouts for the following vectors: st , s , zr , zr_nL , z , zt , $sthat$, $sthatdec$, $bhat$, and $errors$. (You can cause these variables to output to the screen just by removing the semi-colon at the end of the lines where they are computed.) Note that for this part b, the noise vector $w = 0$, and the number of transmitted vectors $n_txd_vectors = 1$. You can comment out or delete the code below the end of the transmitted vectors loop for this part b.
- (c) Now set $n_txd_vectors$ to 500000, and uncomment the line that generates 16 random bits b . Also uncomment and complete the line where the noise vector w is computed. Make sure you have semi-columns at the end of all the lines in the for $k=(1:n_txd_vectors)$ loop, as you do not want anything printed to the screen while the program is running this loop. For this part c, run the simulation, and turn in printouts of the values of the vectors $cerrors$ and $cbers$.
- (d) Fill in the code that computes the $snrs$ vector, and turn in a printout of that vector. The elements of this vector should be numerical ratios, not dBs. Note that this part d can be done independently of parts b and c, as it really only requires that you have part (a) done, and that you use the value $\sigma^2 = 0.5$ in the calculation of the $snrs$.
- (e) Fill in the rest of the code, and turn in the vector of theoretical BERs $thbers$. Does this compare well to the experimental BERs in $cbers$? Also turn in the single number average BER in ber_avg , and the value of the variable BER_ratio . Does this system offer a reasonably uniform BER across all 16 channels? If your answer is yes, briefly explain why; if no, briefly explain why not.

%OFDM BPSK simulation for Lab7

```
clear;
clf;
```

```
sigma2 = 0.5; %Noise variance sigma2 = N0/2.
N = 16; %Number of OFDM channels
h = [1 -0.5 0.3 -0.1 0.4 0.1 -0.05]; %Impulse response
L = length(h) - 1; %Set L, the impulse response length - 1
```

```
%Initialize the DFT matrices (see lecture notes and reading)
WN = exp(-j*2*pi/N);
Nrange = (0:N-1)';
Wexponents = Nrange*Nrange'; %matrix of WN exponents
W = WN.^Wexponents; %DFT forward transform
Wi = ?; %DFT inverse transform (you don't actually need to compute the inverse)
Ws = (1/sqrt(N))*W; %DFT forward transform scaled by 1/sqrt(N)
Wsi = sqrt(N)*Wi; %DFT inverse scaled by sqrt(N)
```

```
%Check the DFT matrices to make sure they multiply to the identity matrix
round(W*Wi)
round(Ws*Wsi)
```

```
he = ?; %Extend h with extra 0s at end to be length N, calling result he
%(Easiest by using vector concatenation and the zeros command)
```

```
he = he.'; %Convert he from 1xN row vector to Nx1 column vector, using non-conjugate transpose
```

```
%Part a
%Compute channel frequency response H[n] using forward DFT matrix W and the he column vector
H = ?
```

```
%Part a
%Make stem plots of abs(H) and angle(H) vs. Nrange
figure(1);
stem(Nrange,abs(H));
ylabel('|H[n]|');
xlabel('n');
figure(2);
stem(Nrange,angle(H));
ylabel('angle(H[n])');
xlabel('n');
```

```
%Start of parts b and c
%Initialize the cumulative channel error count matrix
cerrors = zeros(1,N);
```

```
%Initialize number of symbol vectors to transmit
n_txd_vectors = 1; %For part c you'll set this to 500000
```

```

%Loop to transmit the symbol vectors
for k = (1:n_txd_vectors)

%For part b:
b = [1 0 0 0 0 1 1 1 1 0 1 0 0 1 1 0];

%For part (c), uncomment the following line
%Generate 16 random bits
%b = rand(1,N) < 0.5;

%Shift the bits to BPSK symbols: 0->-1, 1->1
st = ?

%Convert st from 1xN row vector to Nx1 column vector (non-conjugate transpose)
st = ?

%Take IDFT of st using scaled IDFT matrix Wsi
s = ?

%Convert s from column vector to row vector using non-conjugate transpose
s = ?;

%Insert cyclic prefix into s per course notes/reading
%Make sure to insert the cyclic prefix at the START of s,
%so that the first L elements of s are s(N-L+1) to s(N)
%Easy to do this by matrix concatenation using brackets
%Type "help paren" for details. Type "help colon" to find
%out how to pick out selected parts of a vector.
s = ?

%Compute received vector zr by convolving h with s, using MATLAB conv
%command.
zr = conv(s,h);

%For part b:
w = zeros(size(zr));

%For part c: Uncomment and complete the line 6 lines below this one.
%Create complex Gaussian noise vector w with total variance sigma2, of same size as zr
%Make real and imaginary parts have variance sigma2
%Hints: 1) randn produces Gaussian r.v.s with 0 mean and variance 1
%      2) If A is a constant and X an r.v., then var(A*X) = A^2*var(X)

%w = ?*randn(size(zr)) + j*?*randn(size(zr)); %Uncomment and complete for Part c

%Add w to zr, producing zrn
zrn = zr + w;

%Remove cyclic prefix by removing first L samples from zrn, producing zrnL

```

%Hint: type "help colon" to find out how to pick out selected parts of a vector
zrnL = ?

%Keep only first N elements of zrnL; call the result z
z = ?

%Convert z to column vector (non-conjugate transpose)
z = ?

%Take DFT of z using Ws
zt = ?

%Divide zt by H[n] element by element (use ./)
shtat = ?

%Convert shtat to row vector (non-conjugate transpose)
shtat = ?

%Decode elements of shtat to either -1 or +1 by comparing real part of shtat with 0
%Hint: Use the sign() function
shtatdec = sign(?);

%Convert shtatdec into bhat by level shifting from -1->0, 1->1
bhat = ?

%XOR b and bhat, using xor command, to produce an integer variable
%called errors, which has a 1 in each bit position that is in error.
errors = ?

%Update the cumulative total errors in each channel in the cerrors vector
%by adding errors into cerrors
cerrors = cerrors + errors;

end; %end transmitted vectors loop

%End of part b

%Print out channel errors vector cerrors
cerrors

%Compute experimental channel bit error rates for each channel in the cbers vector
cbers = cerrors/?

%Part (d)
%Compute SNRs for each subchannel per course notes, using H and sigma2
%Note that these SNRs should be ratios, not in dB.
snrs = ?

%Part (e)
%Compute theoretical BERs (use BPSK formula); they should agree with cbers
thbers = ?

```
%Print out comparison of experimental and theoretical BERs  
[cbers' thbers]
```

```
%Compute average experimental BER over all channels  
ber_avg = ?
```

```
%Compute ratio of largest to smallest experimental channel BER  
%Hint: use max() and min() functions  
BER_ratio = ?
```